

REVERSE ENGINEERING AN HT1621-BASED LCD SYSTEM

Bit banging|HT1621 protocol & RAM architecture|Driver adaptation (C++)|ESP32 peripheral integration||Hardware reverse-engineering|Embedded programming

Prepared by: Bouhlel Ahmed

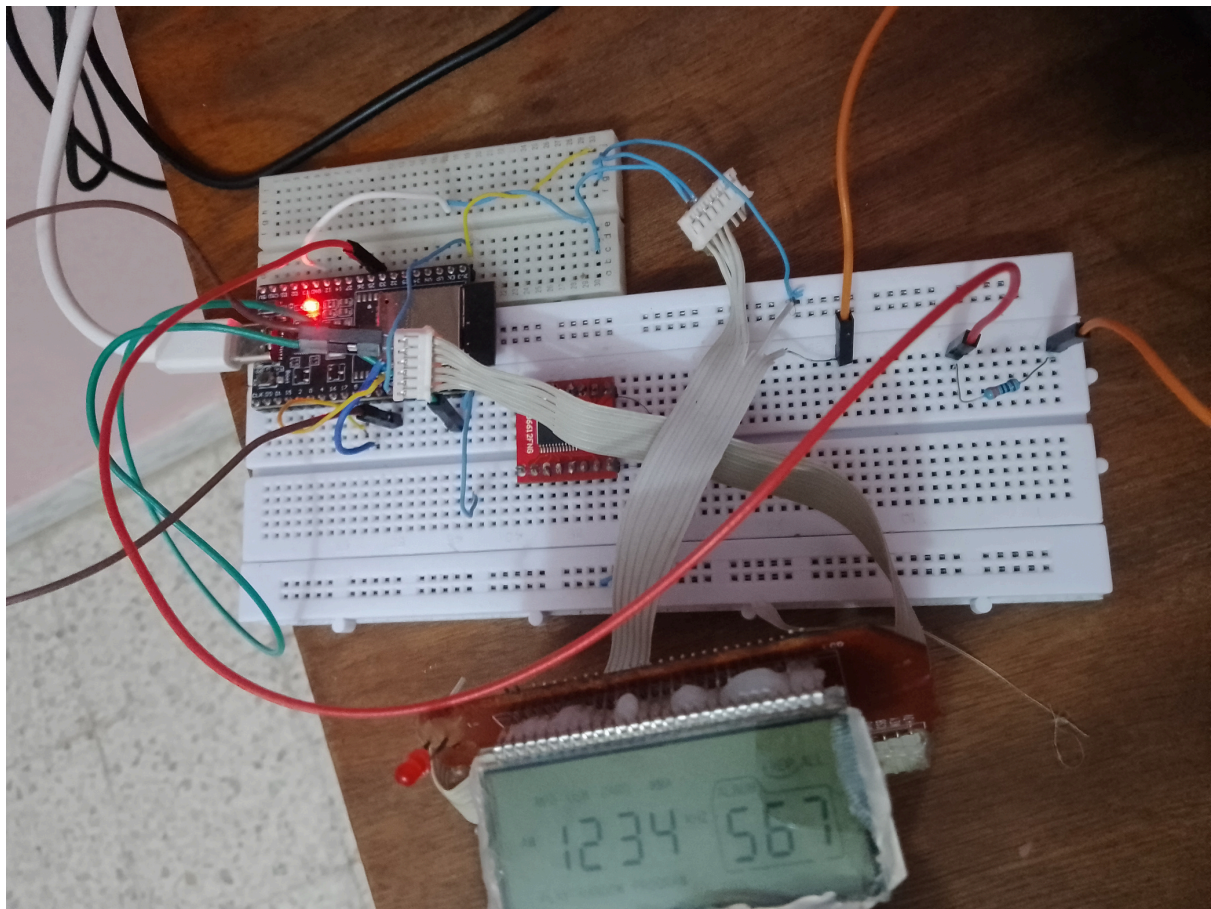
Last updated: july 2026

Overview



This project demonstrates the reverse engineering and reuse of the LCD module from a Thomson RCD-203U radio. The original LCD was driven by two integrated circuits (ET1621 and SC3610). The goal was to understand the hardware architecture, adapt an existing HT1621 driver to the LCD's undocumented memory layout, and successfully control the original display using an ESP32.



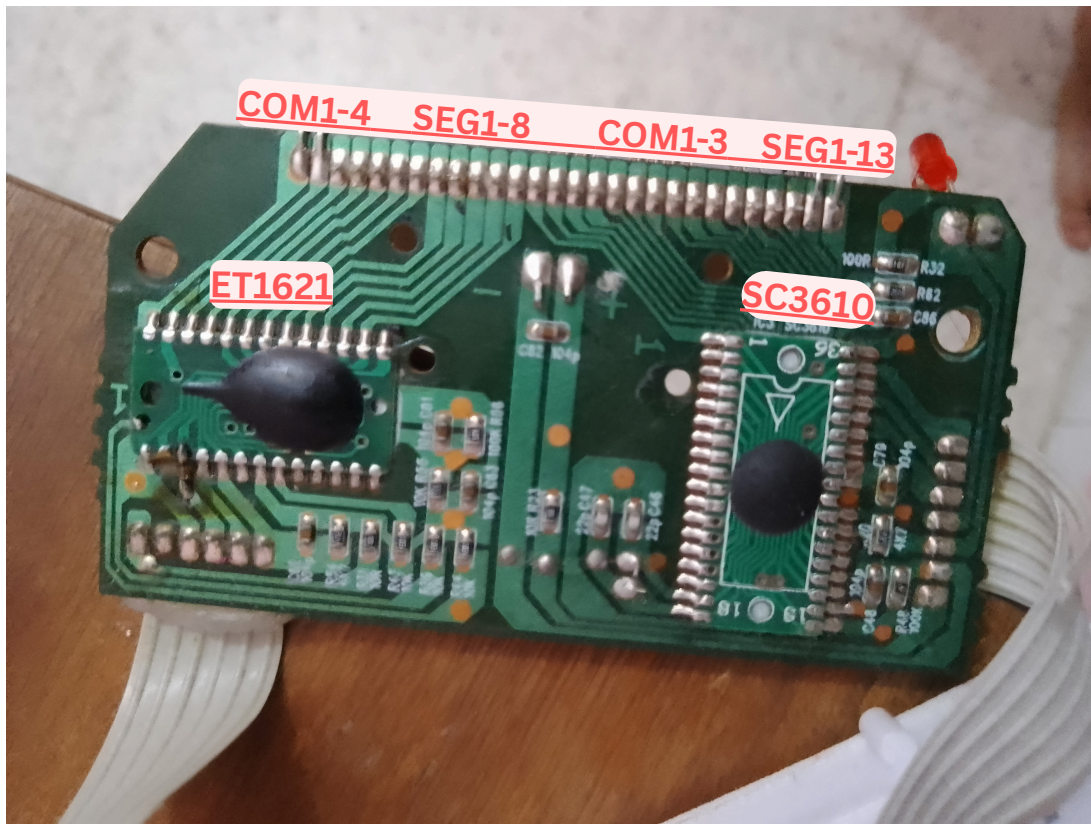


Original Hardware



Component	Function	details
ET1621	LCD driver ic	not all pins r in use
SC3610	Radio Controller + LCD Driver	frequency counter for AM & FM modes,alarm,clock
LCD Glass	Custom segmented display	Custom display with 7 segments and icons
Blue LED	LCD Backlight	powered seperately

LCD Architecture



After searching and studying datasheets, the initially unknown LCD system was found to consist of one LCD glass connected to 2 ICs, with 4 COM wires and 8

SEG wires controlled by ET1621 and 3 COM wires & 13 SEG wires controlled by SC3610, with the ET controlling several icons and a 3-digit 7-segment display whereas the SC controls other icons and a 4-digit 7-segment display.

Initial Challenges



The reverse engineering process began with several unknowns that made the task significantly more complex than simply integrating a standard HT1621 driver:

Identified but Unfamiliar Pins:

The flexible flat cable (FFC) connecting the LCD module to the main board had clearly labeled test points: **DATA**, **WR**, **CS**, **VDD**, and **GND**. However, their exact electrical behavior, voltage levels, and timing requirements were unknown. The labels suggested a serial interface, but no protocol specification was available. (key pin wasn't used)



Unknown Communication Protocol:

The nature of the protocol was unclear—whether SPI, I²C, or a proprietary implementation. The command structure, initialization sequences, and data framing had to be determined from available **datasheets** and **libraries** online

<https://github.com/valerionew/ht1621-7-seg> (library)

<https://www.alldatasheet.com/datasheet-pdf/pdf/64418/HOLTEK/HT1621.html> (HT1621/ET1621 datasheet)

Library Constraints (doesn't display valid numbers):

The available HT1621 library on github was designed for generic 7-digit displays with hardcoded character-to-RAM mappings. It lacked low-level functions for writing to arbitrary RAM addresses and offered no debugging capabilities such as displaying icons or raw bit-pattern testing.



```

1  #include <HT1621.h>
2  #define LCD_CS_PIN 5
3  #define LCD_WR_PIN 18
4  #define LCD_DATA_PIN 23
5
6
7  HT1621 lcd;
8  void setup() {
9      lcd.begin(LCD_CS_PIN, LCD_WR_PIN, LCD_DATA_PIN);
10     lcd.clear();
11 }
12
13 void loop(){
14     lcd.printCelsius(87);
15     delay(2000);
16     lcd.clear();
17     delay(2000);
18 }

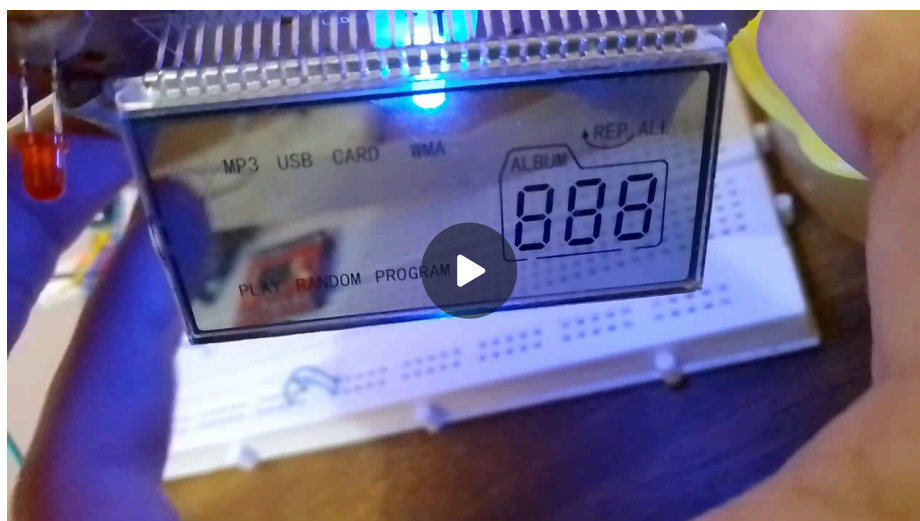
```

Missing LCD Glass Datasheet:

The custom LCD glass had no identifiable part number or documentation. Its internal segment-to-pin mapping—including the arrangement of 7-segment digits, decimal points, and icons—was completely undocumented.

Missing Reflective Diffuser:

The LCD's reflective diffuser was absent from the salvaged unit, significantly reducing contrast. This made visual verification of test patterns difficult under normal lighting. (i glued on my own diffuser later using a white plastic sheet)



Undocumented Memory Layout:

The HT1621's 32×4 -bit RAM (128 bits total) did not follow a logical digit-by-digit organization. The memory addresses corresponding to each display segment were arbitrary and non-contiguous, with some addresses storing icon states and others remaining unused. Which later led to me mapping my own memory from experiments

Multiplexing Complexity:

The HT1621 employs time-division multiplexing across four commons (COMs). Understanding the relationship between COM signals, SEG outputs, and the LCD glass electrode structure required familiarity with multiplexed LCD theory before segment mapping could be accurately performed.

Understanding the HT1621

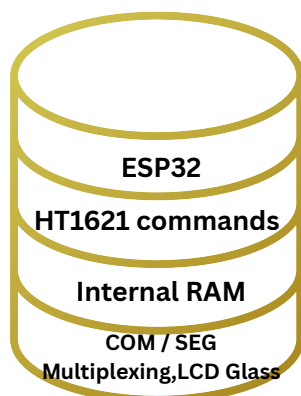


Rather than using the driver as a black box, the complete communication sequence was analyzed.

the ET1621 Contains an integrated 32×4 -bit RAM. You simply write the display data into this memory, its an interface bridge between a microcontroller

(MCU) and a segmented LCD display, offloading the complex task of refreshing the screen. (refreshing needs AC currents because DC causes burn in) which takes us to LCD multiplexing COM lines SEG (each combination of SEG and COM is one segment or icon), bias selection oscillator configuration...

After further studying the library files and the datasheet i was able to construct :



DA: pin: the protocol sends the Most Significant Bit (MSB) first

3 bits for mode

6 bits for address

8 bits for value

101

000010

00100000

WR pin:has a clock for every bit (rising edge)

CS pin:is pulled low when communications starts and high when it ends

Reverse Engineering the LCD



memory mapping procedure

The goal was to remap the memory layout for correct digit display. I started by studying the HT1621 library—from the class structure to its public/private functions handling biasing, digit printing, and low-level bitbanging.

The key discovery was `wrone()`, a private function that sends a frame with a specific address and value to the IC. After making it public, I called it from main and logged every address-value combination to fully map the memory.

```
1  #include <HT1621.h>
2  #define LCD_CS_PIN 5
3  #define LCD_WR_PIN 18
4  #define LCD_DATA_PIN 23
5  char adress=2;
6  char value=b00100000;
7
8  HT1621 lcd;
9  void setup() {
10     lcd.begin(CS_PIN,WR_PIN,DATA_PIN);
11     lcd.clear();
12 }
13
14 void loop(){
15     lcd.wrone(adress,value)
16     delay(2000);
17     lcd.clear();
18     delay(2000);
```

The existing library assumed one

RAM Byte=EntireDigit

However,experimental testing revealed a different organization.

Each RAM byte controlled segments belonging to multiple digits.Example :

RAM Byte :

Segment A of Digit 1

Segment F of Digit 2

Icon Segment C of Digit 3

This meant the original architecture could not correctly support this display **all the functions higher than the wrone are useless and have to be reimplemented** (also only the even adresses were found to be used by the lcd



full segment to memory mapping

(3B means third digit segment B in 7 segment notation)

address	bit	0	1	2	3	4	5	6	7
0		3E	3G	3F	ALL	Program	rep	random	USB
2		2E	2G	2F	play	3D	3C	3B	3A
4		1E	1G	1F		2D	2C	2B	2A
6		card	WMA album	MP3					

Library Modifications



The original library required substantial modifications.

Original Architecture

Character→Segment Pattern→Buffer→RAM→LCD

This architecture assumed one RAM location corresponded to one digit.

That assumption was invalid for our LCD.

New Modified Architecture

Character→Character Lookup→Custom RAM Mapper→HT1621 RAM→LCD

i removed the methods and attributes that weren't compatible for my lcd and created my own

removed :

1. void `print`(long num, const char* flags="%6li", int precision = 0);
2. void `print`(double num, int precision = 3);
3. void `printCelsius`(double num);
4. void `print`(const char* str, bool leftPadded = false);
5. void `setdecimalseparator`(int dposition);
6. char `charToSegBits`(char character);
7. void `setBatteryLevel`(int level);
8. unsigned char `_battery`[3];

added:

1. void `print`(char* text);
2. void `setSegmentbinary`(int addr, int bit, bool state); (bit here 1-7)
3. void `setSegmentInDigit`(int digit, char segment, bool state);
4. void `setDigit`(int digitPosition, char number);
5. void `setDigitnum`(int digitPosition, int num);
6. void `printnum`(int num);
7. void `printicon`(char* text);
8. also modified void `update`() (to fit my number of addresses and buffers)

the one at the lowest level is `setSegmentbinary` which takes care of bitwise OR assignment ,bitwise AND assignment to ensure the buffer's old data isn't corrupted (we only send the data after filling the software buffers)

then `setSegmentInDigit` takes care of adding the abstraction of talking directly about segment names abcdefg and digit number too instead of actual addresses and values

after that `setDigit` and `setdigitnum` takes it even further with only demanding digit number and the number ur trying to display(2 variants for int input and char input which includes letters)

finally `print` and `printnum` display the raw int or char given to them

`printicon` uses directly `setSegmentBinary` to control icons which r just one specific adress and value as the table showed earlier.

examples:

```
lcd.print("bro"), lcd.print("123"), lcd.printnum(456), lcd.printicon("mp3")
```

https://docs.google.com/document/d/1_-BIGvLpNZMgWbOIKv_Vtw0ppX3Eri/edit?usp=sharing&oid=100633524191317493915&rtfpof=true&sd=true (new library

modifications link)

Results&discussion



The project demonstrates that consumer LCD modules can often be reused without replacing the original driver hardware.

The main challenge was not electrical interfacing but understanding the undocumented relationship between

- LCD RAM
- physical segments
- software architecture

This required considerably more reverse engineering than expected. Technical knowledge acquired during the project includes:

- Embedded C++
- Class modification
- Library architecture
- Reverse engineering
- Reading datasheets
- LCD multiplexing
- COM/SEG operation
- HT1621 communication protocol
- Hardware debugging
- Memory mapping
- Bit-level programming /bitbanging



```
HT1621 lcd;
void setup() {
  lcd.begin(CS_PIN,WR_PIN,DATA_PIN);
  lcd.clear();
}

void loop() {
  lcd.print("the");
  delay(2000);
  lcd.print("end");
  delay(2000);
  lcd.print("---");
  delay(2000);
  // Nothing needed
}
```

```
HT1621 lcd;
void setup() {
  lcd.begin(CS_PIN,WR_PIN,DATA_PIN);
  lcd.clear();
  lcd.printicon("mp3");
}

void loop() {
  lcd.printnum(i);
  delay(300);
  i++;
}
```